Ενδιάμεση Εργασία Νευρωνικών Δικτύων – Βαθιάς Μάθησης

Δημήτριος Μουταφτσίδης 9920 (ΤΗΜΜΥ)

moutdimi@ece.auth.gr

Abstract – Η παρούσα εργασία αφορά την υλοποίηση αλγορίθμων ταξινόμησης k-NN και NCC στην βάση δεδομένων CIFAR-10. Περιλαμβάνει σύγκριση μεταξύ των επιδόσεων τους και σχολιασμό αποτελεσμάτων. Θα μελετηθεί , επίσης, η μορφολογία των δεδομένων και η επίδραση του PCA (Principal Component Analysis) στην αποτελεσματικότητα της ταξινόμησης. Τα διαγράμματα και οι πίνακες που παρουσιάζονται αναπαράγονται από το .ipynb αρχείο που επισυνάπτεται.

1. Εισαγωγή

Η CIFAR-10 περιλαμβάνει RGB εικόνες κατανεμημένες σε δέκα κλάσεις. Κάθε κλάση αντιστοιχίζεται σε ένα label 0-9. Περιλαμβάνει διάφορα αντικείμενα και ζώα. Ειδικότερα για το training set:

```
Number of classes:  10

Class labels:  [0 1 2 3 4 5 6 7 8 9]

Actual class names:  ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']

Class-Label correspondance :  {0: 'airplane', 1: 'automobile', 2: 'bird', 3: 'cat', 4: 'deer', 5: 'dog', 6: 'frog', 7: 'horse', 8: 'ship', 9: 'truck'}

Number of instances per class (training set):

  airplane: 5000
automobile: 5000
      bird: 5000
       cat: 5000
      deer: 5000
       dog: 5000
      frog: 5000
     horse: 5000
      ship: 5000
     truck: 5000
```

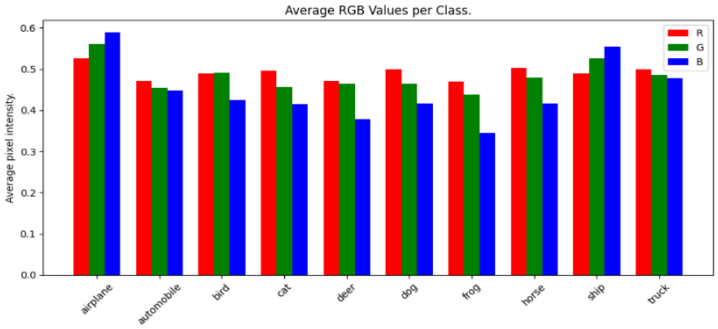
Βλέπουμε πως οι κλάσεις είναι ισάριθμα κατανεμημένες. 5000 εικόνες για κάθε μία. Επίσης:

```
Training data shape:  (50000, 32, 32, 3)

Test data shape:  (10000, 32, 32, 3)

Training labels shape:  (50000,)

Test labels shape:  (10000,)
```

Τα δεδομένα έχουν μορφή numpy array τεσσάρων διαστάσεων. 50.000 εικόνες διαστάσεων 32x32 pixels με τρεις θέσεις για τις RGB τιμές. Τα labels είναι διάνυσμα μήκους 50.000. Όλα τα δείγματα αντιστοιχούν σε κάποιο. Το dataset είναι πλήρες και καλώς ορισμένο. Τέλος, γνωρίζουμε πως οι εικόνες λαμβάνουν τιμές στο διάστημα [0-255] για κάθε κανάλι. Κανονικοποιούμε στο διάστημα [0,1] και μετατρέπουμε σε float32 για αριθμητική ακρίβεια.

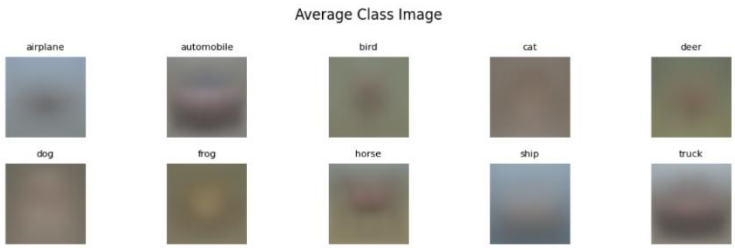
2. Εξέταση Χρωματικού περιεχομένου εικόνων.

Η περαιτέρω ανάλυση του περιεχομένου των εικόνων πιθανότατα προσφέρει χρήσιμα insights για την λειτουργία των αλγορίθμων. Το παρακάτω διάγραμμα απεικονίζει την μέση τιμή όλων των εικόνων της κάθε κλάσης για κάθε RGB κανάλι. Οριμένες κλάσεις έχουν



παρόμοια χρωματικά προφίλ. Θα ήταν ενδιαφέρον να δούμε πως κάτι τέτοιο μπορεί να επηρρεάσει την ταξινόμηση. Ας δούμε πως μοιάζει η παραπάνω μέση χρωματική σύνθεση. Το ακόλουθο διάγραμμα απεικονίζει την σύνθεση των παραπάνω μέσων τιμών RGB για κάθε κλάση.

Δημιουργήσαμε επίσης διάγραμμα της «μέσης εικόνας» που αντιστοιχεί σε κάθε κλάση. Πρόκειται για average RGB τιμών για κάθε εικόνα της κλάσης,



διατηρώντας όμως την θέση του κάθε pixel.

Ενδιαφέρον είναι το πως σε κλάσεις όπως το Άλογο η γενική δομή διατηρείται. Από την άλλη, στην κλάση Σκυλί ,για παράδειγμα, βλέπουμε έντονο skewing. Δεν ισχυριζόμαστε πως η ερμηνεια των αποτελεσμάτων εξηγείται πλήρως από τα παραπάνω.

3. Αξιολόγηση Επίδοσης Ταξινομητή.

Για την επίδοση των αλγορίθμων στο test set χρησιμοποιούμε τα built-in metrics της sickitlearn. Πρώτο και κυριότερο το Classification Report. Περιλαμβάνει τρία χρήσιμα metrics. Το precision

ορίζεται ως το κλάσμα $\frac{True\ Positives}{True+False\ Positives}$. Απλούστερα: Από όλα τα X που ανιχνεύτηκαν , πόσα ήταν πράγματι X; Δηλαδή, όταν ανιχνεύουμε X, πόσο πιθανό είναι να είναι όντως X; Το recall ορίζεται ως $\frac{True\ Positives}{All\ Class\ Instances}$. Με άλλα λόγια: από όλα τα Y που υπάρχουν, πόσα ανιχνεύτηκαν επιτυχώς; Τέλος, για μια εξισορροπημένη αξιολόγηση επίδοσης ανατρέχουμε στο F1 score. $F1 = 2\frac{Precision \cdot Recall}{Precision+Recall}$. Προσφέρει συνολικότερη εικόνα. Υψηλό F1 score υποδεικνύει και αξιόπιστη και συχνή ανίχνευση/ορθή ταξινόμηση. Δεύτερο χρήσιμο εργαλείο αποτελεί ο Confusion Matrix. Στον ένα άξονα έχουμε τα πραγματικά και στον άλλο τα προβλεπόμενα labels. Οι θέσεις του αντιστοιχούν στα δείγματα κλάσης X, Y, Z, … που ταξινομήθηκαν ως X, Y, Z,… αντίστοιχα. Τα True Positives βρίσκονται στην διαγώνιο. Είναι χρήσιμος γιατί μας πληροφορεί για το ποιες κλάσεις «μπερδεύονται μεταξύ τους».

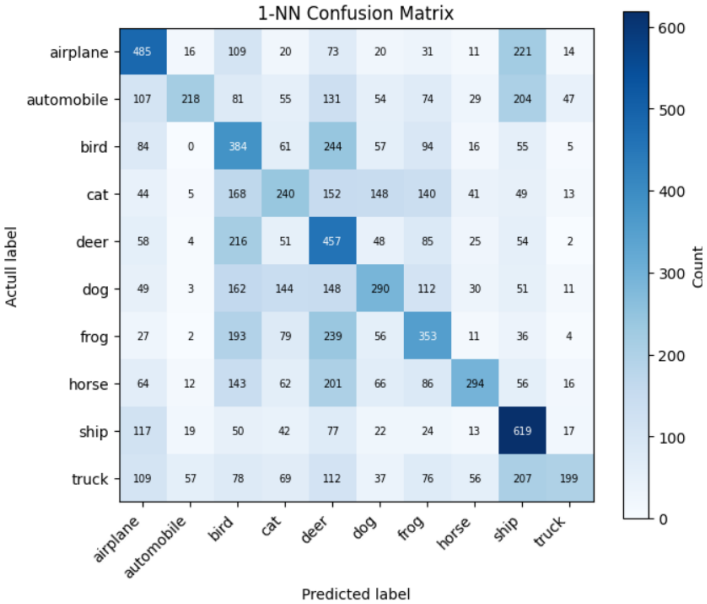
4. k-NN Classifier

Πρόκειται για **μη παραμετρική** μέθοδο. Δεν δημιουργεί μοντέλο προσαρμόζοντας παραμέτρους όπως βάρη και σταθερές. Το μοντέλο *είναι* ,στην ουσία, τα ίδια τα training data. Κατά την εκπαίδευση του, δημιουργεί έναν υπερχώρο με τα labeled training samples. Όταν λάβει ένα προς ταξινόμηση δείγμα, υπολογίζει του k κοντινότερους γείτονες. Το αναθέτει στην κλάση που πλειοψηφεί. Για τον έλεγχο γειτνίασης χρησιμοποιείται συνήθως η **Ευκλείδια Απόσταση** μεταξύ των samples. $\boldsymbol{D(x, y)} = \sqrt{\sum_{k=1}^{n}(x_k - y_k)^2}$. Για τις ανάγκες αυτής της εργασίας τα x, y είναι διανύσματα διαστάσεων $n = 32 \cdot 32 \cdot 3 = 3072$. Πρόκειται για τις flattened εκδοχές των εικόνων ως προς τις θέσεις των pixels και των τριάδων RGB. Ένας ακόμα τρόπος είναι το **Cosine Distance.** Αγνοεί τα magnitudes των διανυσμάτων και συγκρίνει με βάση την κατεύθυνση τους. $\boldsymbol{D(x, y) = 1 - \frac{x \cdot y}{||x|| \cdot ||y||} = 1 - cos(\theta)}$. Έτσι, εξουδετερώνει την επίδραση της διασποράς φωτεινότητας μεταξύ των εικόνων. Εάν παίρναμε μια σκοτεινότερη εκδοχή της ίδιας εικόνας, η Ευκλείδια Απόσταση θα είχε μη αμελητέα τιμή. Η Cosine Distance, από την άλλη, θα ήταν ουσιαστικά μηδενική.

4.1.1. 1-NN Classifier (Eucledean) – Αποτελέσματα.

Για την εφαρμογή του αλγορίθμου καλείται η ομόνυμη built-in συνάρτηση της sickitlearn. Πρωτού περάσουμε τα δεδομένα ως όρισμα, τα κάνουμε flatten σε δύο διαστάσεις. Πλέον δουλεύουμε με numpy array διαστάσεων 50.000 x 3072. Τα pixel positions και τα RGB κανάλια στοιχίζονται σε ένα διάνυσμα. Αυτό ισχύει για όλους τους ταξινομητές που θα χρησιμοποιήσουμε. Ακολουθούν τα αποτελέσματα.

```
1-NN accuracy: 0.3539

Training time for 1-NN:  0.4828512668609619

Classification time for 1-NN:  23.50261616706848

Detailed classification report for 1-NN:

              precision    recall  f1-score   support

    airplane       0.42      0.48      0.45      1000
  automobile       0.65      0.22      0.33      1000
        bird       0.24      0.38      0.30      1000
         cat       0.29      0.24      0.26      1000
        deer       0.25      0.46      0.32      1000
         dog       0.36      0.29      0.32      1000
        frog       0.33      0.35      0.34      1000
       horse       0.56      0.29      0.39      1000
        ship       0.40      0.62      0.49      1000
       truck       0.61      0.20      0.30      1000

    accuracy                           0.35     10000
   macro avg       0.41      0.35      0.35     10000
weighted avg       0.41      0.35      0.35     10000
```



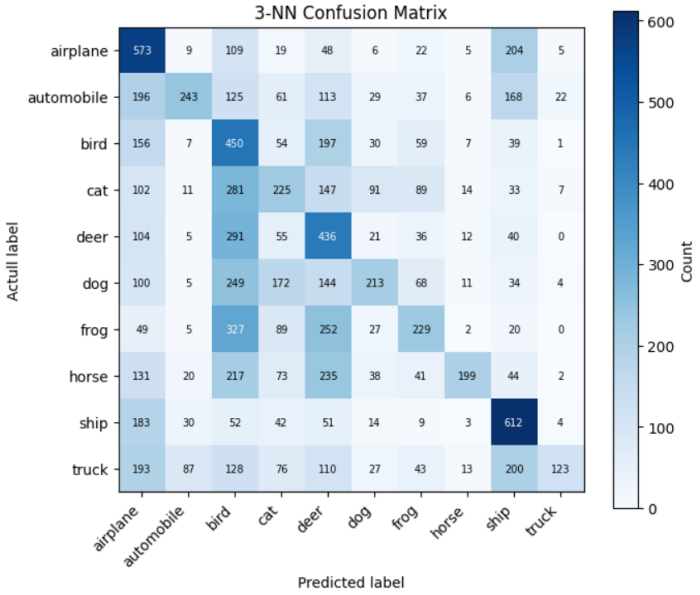
Η επίδοση είναι γενικά χαμηλή. Της τάξεως του 35%. Παρατηρούμε πως κάθε κλάση δεν ταξινομείται εξίσου καλά. Έχουμε «δυνατές» κλάσεις όπως Πλοίο και Αεροπλάνο. Επίσης έχουμε «αδύναμες» όπως Γάτα και Πουλί. Εδώ καταλαβαίνουμε την χρησιμότητα του F1 score ως True Performance Metric. Ας δούμε για παράδειγμα την κλάση Αυτοκίνητο. Έχει το υψηλότερο precision. Δηλαδή όταν ανιχνεύεται, κατά πάσα πιθανότητα όντως είναι. Έχει όμως πολύ χαμηλό recall. Λίγα Αυτοκίνητα ανιχνεύονται πράγματι. Έτσι καταλήγει να είναι σχετικά αδύναμη κλάση. Μία επίσης ενδιαφέρουσα παρατήρηση προκύπτει συνδυάζοντας το Confusion Matrix με το RGB

ραβδόγραμμα της Ενότητας 1. Τα missclassifications της κλάσης Αεροπλάνο αφορούν κυρίως την κλάση Πλοίο και αντίστροφα. Παρατηρείται και σε μερικές κλάσεις ακόμα (ας τις παραλείψουμε). Ανατρέχοντας στο ραβδόγραμμα βλεπουμε σχεδόν ίδιο χρωματικό προφίλ. Θα περιμέναμε λοιπόν το μοτίβο να επαλαμβάνεται και για τις κλάσεις Αυτοκίνητο και Φορτηγό. Κάτι τέτοιο δεν ισχύει. Παρότι, λοιπόν, η παραπάνω σύνδεση μας κινεί το ενδιαφέρον, χρειάζεται σίγουρα αναλυτικότερη διερεύνηση.

4.1.2. 3-NN Classifier (Eucledean) – Αποτελέσματα.

Πλέον ζητούμε από την sickitlearn να λάβει υπόψην τρεις «γείτονες». Ειδού τα αποτελέσματα:

```
3-NN accuracy: 0.3303

Training time for 3-NN:  0.22865056991577148

Classification time for 3-NN:  21.84606146812439

Detailed classification report for 3-NN:

              precision    recall  f1-score   support

    airplane       0.32      0.57      0.41      1000
  automobile       0.58      0.24      0.34      1000
        bird       0.20      0.45      0.28      1000
         cat       0.26      0.23      0.24      1000
        deer       0.25      0.44      0.32      1000
         dog       0.43      0.21      0.28      1000
        frog       0.36      0.23      0.28      1000
       horse       0.73      0.20      0.31      1000
        ship       0.44      0.61      0.51      1000
       truck       0.73      0.12      0.21      1000

    accuracy                           0.33     10000
   macro avg       0.43      0.33      0.32     10000
weighted avg       0.43      0.33      0.32     10000
```

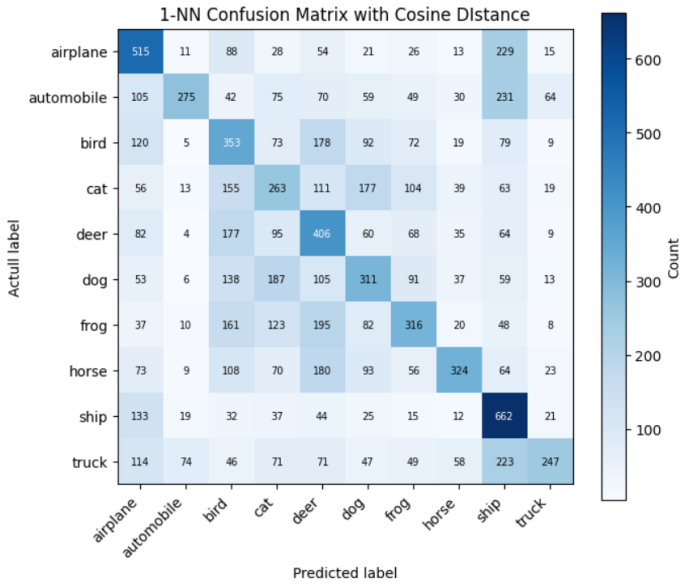


Η ακρίβεια και το training time κινούνται σε αντίστοιχα επίπεδα με προηγούμενως. Μάλιστα παρατηρούμε μικρή μείωση επίδοσης. Μπορούμε λοιπόν να πούμε πως η αύξηση του αριθμού των «γειτόνων» δεν συνεπάγεται απαραίτητα με αυξημένη ακρίβεια. Η ιεραρχική δομή της επίδοσης κάθε κλάσεις φαίνεται να διατηρείται. Υπάρχουν μερικές διαφοροποιήσεις, χωρίς όμως κάποια ιδιαίτερη σημασία. Η κλάση Φορτηγό , παραδείγματος χάρη, είναι πλεόν τελευταία έναντι της κλάσης Γάτα που βλέπαμε προηγουμένως. Δεν έχει, ωστόσο σημαντική ιεραρχική βελτίωση. Ας μην επεκταθούμε παραπάνω. Συνοψίζουμε λοιπόν στο εξής **συμπέρασμα** για της ενότητες 4.1.1. και 4.1.2. : ο αριθμός των γειτόνων για το δεδομένο dataset δεν φαίνεται να έχει σημαντική επίδραση στα αποτελέσματα της ταξινόμησης.

4.2.1. k-NN Classifier (Cosine Distance) – Αποτελέσματα.

Δοκιμάστηκαν και οι δύο εκδοχές για k = 1 , 3. Τόσο στον κώδικα, όσο και στην αναφορά κρατάμε τον 1-NN, καθώς πάλι σημειώθηκε ελαφρώς καλύτερη επίδοση. Ακολουθούν τα αποτελέσματα:

```
1-NN accuracy: 0.3672

Training time for 1-NN:  0.43323659896850586

Classification time for 1-NN:  12.191648721694946

Detailed classification report for 1-NN with Cosine Distance:

              precision    recall  f1-score   support

    airplane       0.40      0.52      0.45      1000
  automobile       0.65      0.28      0.39      1000
        bird       0.27      0.35      0.31      1000
         cat       0.26      0.26      0.26      1000
        deer       0.29      0.41      0.34      1000
         dog       0.32      0.31      0.32      1000
        frog       0.37      0.32      0.34      1000
       horse       0.55      0.32      0.41      1000
        ship       0.38      0.66      0.49      1000
       truck       0.58      0.25      0.35      1000

    accuracy                           0.37     10000
   macro avg       0.41      0.37      0.36     10000
weighted avg       0.41      0.37      0.36     10000
```



Παρατηρούμε μικρή βελτίωση στην απόδοση του αλγορίθμου. Χωρίς ωστόσο σημαντική διαφορά. Πάλι

έχουμε μικρές μεταβολές στα metrics της κάθε κλάσης. Η συνολικλη ιεραρχία, ωστόσο, διατηρείται σε σχέση με τον αρχικό 1-NN. Το **συμπέρασμα** για την ενότητα αυτή λοιπόν είναι: Παρατηρείται μικρή βελτίωση στην επίδοση, χωρίς ωστόσο αξιόλογες διαφορές.

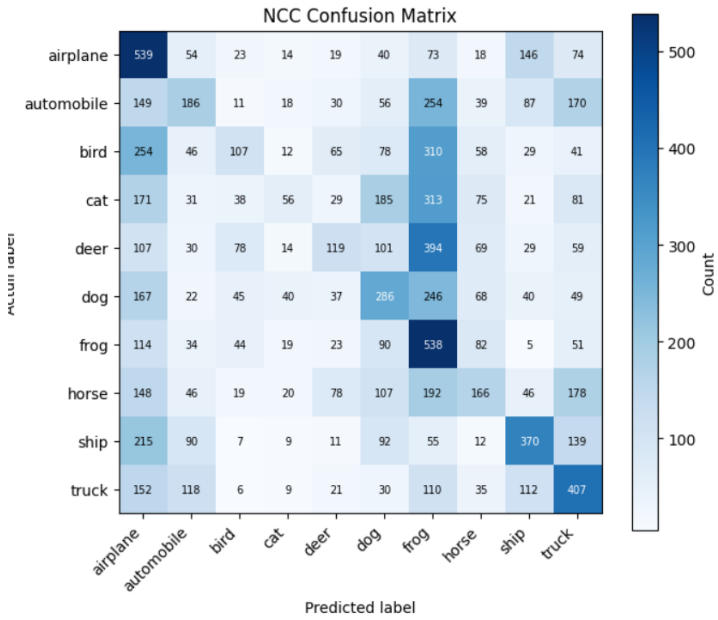
5. NC Classifier.

Πρόκειται για τον **παραμετρικό** «ξάδελφο» του k-NN. Η προς εκπαίδευση παράμετρος είναι τα Κέντρα (Centroids) της κάθε κλάσης. Ο αλγόριθμος υπολογίζει το mean feature vector για κάθε κλάση σύμφωνα με την σχέση: $\boldsymbol{C_c} = \frac{1}{N_c}\sum_{i=1}^{N_c} \boldsymbol{x_i}$, για κάθε κλάση c με N_c δείγματα. Επομένως, μετά την φάση της εκπαίδευσης καταλήγουμε με **c Centroids.** Αυτό είναι το ολοκληρωμένο μοντέλο μας. Στην φάση της ταξινόμησης συγκρίνεται η απόσταση των δειγμάτων από τα Centroids του μοντέλου. Επιλέγεται η πλησιέστερη κλάση. Ως μέτρο εγγύτητας χρησιμοποιείται συνήθως η **Ευκλείδια Απόσταση**. Υπάρχει και η Απόσταση Manhattan. Ορίζεται ως εξής: $\boldsymbol{D(x\,,y)} = \sum_{k=1}^{n} |\boldsymbol{x_k} - \boldsymbol{y_k}|$. Λόγω της απλής αριθμητικής που εμπλέκεται στην φάση εκπαίδευσης ο αλγόριθμος τίνει να είναι πολύ γρήγορος. Επίσης πλεονεκτεί στην ταχύτητα και στην φάση ταξινόμησης, αφού χρειάζεται να πραγματοποιείσει πολύ λιγότερες συγκρίσεις.

4.1. NCC – Αποτελέσματα.

Ακολουθούν τα αποτελέσματα για NCC με Eucledean Metric. Η απόσταση Manhattan δοκιμάστηκε αλλά παραλείπεται.

```
Nearest Centroid accuracy:  0.2774

Training time for NCC:  0.881990909576416

Classification time NCC:  0.32459521293640137

Classification report for Nearest Centroid:

              precision    recall  f1-score   support

    airplane       0.27      0.54      0.36      1000
  automobile       0.28      0.19      0.22      1000
        bird       0.28      0.11      0.16      1000
         cat       0.27      0.06      0.09      1000
        deer       0.28      0.12      0.17      1000
         dog       0.27      0.29      0.28      1000
        frog       0.22      0.54      0.31      1000
       horse       0.27      0.17      0.20      1000
        ship       0.42      0.37      0.39      1000
       truck       0.33      0.41      0.36      1000

    accuracy                           0.28     10000
   macro avg       0.29      0.28      0.25     10000
weighted avg       0.29      0.28      0.25     10000
```



Θα επαναλάβουμε για ακόμη μια φορά το σχόλιο περί διατήρησης της ιεραρχίας των κλάσεων. Η συνολική επίδοση είναι, ωστόσο, σημαντικά χαμηλότερη. Το πραγματικά εντυπωσιακό είναι το Classification Time. Είναι δεκάδες φορές μικρότερο σε σύγκριση με τους k-NN. Ως **συμπέρασμα** κρατάμε πως: ο NCC θυσιάζει ακρίβεια για σημαντικά μικρότερο χρόνο εκτέλεσης.

6. PCA & Βελτίωση Επίδοσης.

Πρόκειται για τεχνική μείωσης των διαστάσεων των δεδομένων. Χρησιμοποιείται συχνά για την βελτίωση επίδοσης διαφόρων ταξινομητών. Σκοπός είναι η απαλοιφή των ισχυρά συσχετιζόμενων features. Διατηρούνται τα (όσο το δυνατόν) ασυσχέτιστα που ευθύνονται για το μεγαλύτερο μέρος της διασποράς. Αυτά καλούνται Principal Components. Απλούστερα, απομονώνονται όσο δυνατόν λιγότερα fetaures που περιέχουν όσο δυνατόν περισσότερη πληροφορία για τα δεδομέναΗ λειτουργία του συνοψίζεται σε τρία βήματα:

1) **Demeaning.** Θέλουμε δεδομένα μηδενικής μέσης τιμής.
2) **Υπολογισμός Covariance Matrix.** Περιέχει την πληροφορία συχέτισης μεταξύ των features.
3) **Υπολογισμός ιδιοδιανυσμάτων και ιδιοτιμών.** Τα πρώτα υποδεικνύουν τις κατευθύνσεις μέγιστου variance και οι δεύτερες το πόσο από αυτό περιέχεται.

Επιλέγεται, στην συνέχεια, κάποιος αριθμός από τα κυρίαρχα features με βάση την ιεραρχία των ιδιοτιμών τους. Έτσι, αφαιρούνται περιττά features και θόρυβος. Ο χρόνος εκπαίδευσης μειώνεται και αποφεύγουμε το overfitting.

6.1. PCA και επίδοση - Grid Search.

Για την διερεύνηση αποτελεσματικότητας του PCA πραγματοποιούμε Grid Search με διάφορες τιμές για τα Principal Components. Το εφαρμόζουμε επίσης σε όλα τα μοντέλα που είδαμε μέχρι στιγμής. Θεωρήσαμε πως είναι πιο ενδιφέρον από το να το εφαρμόζαμε στο αποτελεσματικότερο μοντέλο εξ'αρχής. Έτσι θα έχουμε την ευκαιρία θα διερευνήσουμε την επίδραση που έχει σε κάθε μοντέλο. Ακολουθούν τα αποτελέσματα:

```
Results of Grid Search:

PCA = PCA_80      |  1-NN Cos    |  acc=0.4234
PCA = PCA_90      |  1-NN Cos    |  acc=0.4226
PCA = PCA_60      |  1-NN Cos    |  acc=0.4222
PCA = PCA_120     |  1-NN Cos    |  acc=0.4220
PCA = PCA_200     |  1-NN Cos    |  acc=0.4206
PCA = PCA_90      |  3-NN Cos    |  acc=0.4184
PCA = PCA_50      |  1-NN Cos    |  acc=0.4167
PCA = PCA_120     |  3-NN Cos    |  acc=0.4167
PCA = PCA_80      |  3-NN Cos    |  acc=0.4164
PCA = PCA_60      |  3-NN Cos    |  acc=0.4143
PCA = PCA_200     |  3-NN Cos    |  acc=0.4129
PCA = PCA_50      |  3-NN Cos    |  acc=0.4086
PCA = PCA_60      |  1-NN Euc    |  acc=0.3943
PCA = PCA_50      |  1-NN Euc    |  acc=0.3892
PCA = PCA_80      |  1-NN Euc    |  acc=0.3881
PCA = PCA_90      |  1-NN Euc    |  acc=0.3880
PCA = PCA_20      |  3-NN Cos    |  acc=0.3852
PCA = PCA_120     |  1-NN Euc    |  acc=0.3822
PCA = PCA_20      |  1-NN Cos    |  acc=0.3768
PCA = PCA_50      |  3-NN Euc    |  acc=0.3761
PCA = PCA_60      |  3-NN Euc    |  acc=0.3752
PCA = PCA_20      |  1-NN Euc    |  acc=0.3735
PCA = PCA_200     |  1-NN Euc    |  acc=0.3734
PCA = PCA_80      |  3-NN Euc    |  acc=0.3719
PCA = PCA_90      |  3-NN Euc    |  acc=0.3714
PCA = PCA_20      |  3-NN Euc    |  acc=0.3691
PCA = PCA_120     |  3-NN Euc    |  acc=0.3595
PCA = PCA_200     |  3-NN Euc    |  acc=0.3534
PCA = PCA_200     |  NCC         |  acc=0.2773
PCA = PCA_120     |  NCC         |  acc=0.2767
PCA = PCA_90      |  NCC         |  acc=0.2765
PCA = PCA_80      |  NCC         |  acc=0.2763
PCA = PCA_50      |  NCC         |  acc=0.2760
PCA = PCA_60      |  NCC         |  acc=0.2760
PCA = PCA_20      |  NCC         |  acc=0.2732
```

Η βέλτιστη επίδοση επιτυγχάνεται από τον 1-NN με PCA για 80 components. Η γενικότερη επίδοση βελτιώθηκε σημαντικά. Παρατηρούμε επίσης πως τα 1-NN με Cosine Distance Metric κυριαρχούν. Ακολουθούν τα 3-NN με Cosine, ξεπερνώντας σε γενικές γραμμές τα 1-NN Eucledean. Σημειώνουμε πως το 1-NN Cosine με 20 Components αποδίδει χειρότερα από το 1-NN Eucledean με 120 (ένα από τα διάφορα παραδείγματα). Νωρίτερα είχαμε δει πως το Cosine Distance δίνει καλύτερα αποτελέσματα.

Οδηγούμαστε έτσι στο ακόλουθο **συμπέρασμα:** το PCA δύναται να ενισχύσει αλλά και να βλάψει την επίδοση. Η κατάλληλη επιλογή του αριθμού των components είναι κρίσιμη. Πάρα πολλά αμβλύνουν την επίδρασή του. Πολύ λίγα «σαμποτάρουν» το μοντέλο.

Τέλος, να προσθέσουμε πως το PCA αφήνει τον NCC σχεδόν ανεπηρρέαστο. Κάτι τέτοιο έχει λογική, καθώς και οι δύο μέθοδοι βασίζονται σε μέσες τιμές των δεδομένων. Το PCA είναι ουσιαστικά γραμμικός μετασχηματισμός. Δεν αλλοιώνει τη σχετική διάταξη των centroids, τα αποτελέσματα του NCC παραμένουν σχεδόν τα ίδια.

7. Διάφορα – Προσωπικό Σημείωμα

Για την βελτίωση της απόδοσης των ταξινομητών προσπάθησα να χρησιμοποιήσω διάφορες κλασσικές τεχνικές Ψηφιακής Επεξεργασίας Εικόνας. Είναι από τα αγαπημένα μου μαθήματα και συνδέεται άψογα με Image Classification Projects. Αρχικά προσπάθησα να εφαρμόσω τελεστές Sobel για ανίχνευση ακμών. Αποσκοπούσα στον διαχωρισμό των εικόνων από τα background. Απέτυχα παταγωδώς. Οι εικόνες έιναι πολύ μικρές. Έκανα επίσης δοκιμές σε grayscale , εφάρμοσα δίορθωση ιστογράμματος και πειραματίστηκα με διόρθωση χρώματος και φωτεινότητα. Δεν είδα κανένα αξιόλογο αποτέλεσμα. Έτσι επέλεξα να μην συμπεριλάβω τις εν λόγω δοκιμές ούτε σε επίπεδο κώδικα (έτσι κι αλλίως οι περισσότεροι αλγόριθμοι ήρθαν από εργασία μου στο μάθημα Ψ.Ε.Ε. του ΤΗΜΜΥ, οπότε δεν χρειάστηκε να τους γράψω εκ νέου) ούτε σε επίπεδο αποτελεσμάτων. Παραθέτω το παρόν σημείωμα κυρίως για λόγους πληρότητας και ως προ-εισαγωγή σε περίπτωση που κάποια από αυτές τις τεχνικές λειτουργήσει στο CNN του επόμενου παραδοτέου.